

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И
ИНФОРМАТИКИ

Кафедра «Программная инженерия»

Лабораторная работа № N6
по дисциплине «Информационные технологии и программирование»
по теме: «Разработка веб-приложения на Spring Boot. Тестирование»

Выполнил студент группы:
БПИ2401 Исламов Эмин Маратович
Проверил: Харрасов Камиль
Раисович

Москва 2026

Цель работы

Изучить основы тестирования Java/Spring Boot-приложений: написать unit-тесты сервисов с изоляцией зависимостей через Mockito, применить verify/spy, а также выполнить базовый web-тест контроллера через @WebMvcTest.

Краткая теория

Зачем тесты в Spring Boot

Spring Boot-приложение состоит из нескольких слоёв: controller (HTTP), service (бизнес-логика), repository (данные), security (доступ). В зависимости от слоя выбирают тип теста: unit / integration / web.

JUnit

JUnit даёт: @Test, lifecycle-аннотации (@BeforeEach, @AfterEach), набор assertions (assertEquals, assertNotNull, assertThrows и т.д.).

Mockito

Mockito - библиотека для mock-объектов (заглушек зависимостей). Ключевые инструменты: @Mock, @InjectMocks, when(...).thenReturn(...), verify(...), spy(...).

Ход выполнения работы

Часть 0. Подготовка проекта

Использован проект spring-lab3-notifications, расширенный в ЛР №4 и №5. Проверено наличие тестовой зависимости Spring Boot spring-boot-starter-test со scope=test.

Фрагмент pom.xml:

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-test</artifactId>
```

```
<scope>test</scope>  
</dependency>
```

Часть 1–2. Базовые тесты JUnit (пример структуры)

Создан простой тест SimpleTest и/или демонстрация assertEquals, assertNotNull, assertTrue, assertThrows.

Листинг: src/test/java/org/example/SimpleTest.java

```
public class SimpleTest {  
    @Test  
    void shouldAddNumbers() {  
        int result = 1 + 2;  
        assertEquals(3, result);  
    }  
}
```

Часть 3–7. Unit-тесты сервисов (Mockito)

3.1 Unit-тест UserService.getUserId()

Листинг: src/test/java/org/example/service/UserServiceGetByIdTest.java

```
@ExtendWith(MockitoExtension.class)  
class UserServiceGetByIdTest {  
    @Mock  
    private UserRepository userRepository;  
    @InjectMocks  
    private UserService userService;  
    @Test  
    void shouldReturnUserById() {  
        User user = new User();  
        user.setId(1L);  
        user.setEmail("ivan@test.com");  
    }  
}
```

```

        user.setName("Иван");

        when(userRepository.findById(1L)).thenReturn(Optional.of(user));

        User result = userService.getUserById(1L);

        assertNotNull(result);

        assertEquals(1L, result.getId());

        assertEquals("ivan@test.com", result.getEmail());
    }
}

```

3.2 Unit-тест UserService.deleteUser() + verify(delete)

Проверено, что вызывается userRepository.delete(...).

Листинг: src/test/java/org/example/service/UserServiceDeleteTest.java

```

@ExtendWith(MockitoExtension.class)
class UserServiceDeleteTest {

    @Mock
    private UserRepository userRepository;

    @InjectMocks
    private UserService userService;

    @Test
    void shouldDeleteUserAndCallRepositoryDelete() {
        User user = new User();
        user.setId(10L);

        when(userRepository.findById(10L)).thenReturn(Optional.of(user));

        userService.deleteUser(10L);

        verify(userRepository, times(1)).delete(user);
    }
}

```

3.3 Unit-тест NotificationService.getNotificationById()

Листинг: src/test/java/org/example/service/NotificationServiceGetByIdTest.java

```
@ExtendWith(MockitoExtension.class)
class NotificationServiceGetByIdTest {
    @Mock
    private NotificationRepository notificationRepository;
    @Mock
    private UserRepository userRepository;
    @InjectMocks
    private NotificationService notificationService;
    @Test
    void shouldReturnNotificationById() {
        Notification n = new Notification();
        n.setId(5L);
        n.setTitle("Test");
        when(notificationRepository.findById(5L)).thenReturn(Optional.of(n));
        Notification result = notificationService.getNotificationById(5L);
        assertNotNull(result);
        assertEquals(5L, result.getId());
        assertEquals("Test", result.getTitle());
    }
}
```

3.4 Негативный тест: уведомление не найдено (assertThrows)

Методичка прямо показывает такой тип теста для ошибок.

Листинг: src/test/java/org/example/service/NotificationServiceNotFoundTest.java

```

@ExtendWith(MockitoExtension.class)
class NotificationServiceNotFoundTest {
    @Mock
    private NotificationRepository notificationRepository;
    @Mock
    private UserRepository userRepository;
    @InjectMocks
    private NotificationService notificationService;
    @Test
    void shouldThrowWhenNotificationNotFound()
    { when(notificationRepository.findById(999L)).thenReturn(Optional.empty());
      assertThrows(RuntimeException.class, () ->
notificationService.getNotificationById(999L));
    }
}

```

Часть 5. Отдельный класс тестов для AuthService (из ЛР по Security)

Листинг: src/test/java/org/example/service/AuthServiceTest.java

```

@ExtendWith(MockitoExtension.class)
class AuthServiceTest {
    @Mock
    private UserRepository userRepository;
    @Mock
    private PasswordEncoder passwordEncoder;
    @InjectMocks
    private AuthService authService;
}

```

```

@Test
void
registerShouldSaveUserWithEncodedPasswordAndUserRole() {
    RegisterRequest req = new RegisterRequest();
    req.setName("Emin");
    req.setEmail("emin@test.com");
    req.setPassword("123456");
    when(userRepository.existsByEmail("emin@test.com")).thenReturn(false);
    when(passwordEncoder.encode("123456")).thenReturn("bcrypt_hash");

    authService.register(req);

    ArgumentCaptor<User> captor =
ArgumentCaptor.forClass(User.class);

    verify(userRepository,
times(1)).save(captor.capture());

    User saved = captor.getValue();

    assertEquals("Emin", saved.getName());
    assertEquals("emin@test.com", saved.getEmail());
    assertEquals("bcrypt_hash", saved.getPassword());
    assertEquals(UserRole.ROLE_USER, saved.getRole());
    assertNotNull(saved.getCreatedAt());

    verify(passwordEncoder, times(1)).encode("123456");
    verify(userRepository,
times(1)).existsByEmail("emin@test.com");
}

@Test
void registerShouldThrowConflictWhenEmailExists() {
    RegisterRequest req = new RegisterRequest();

```

```

        req.setName("Emin");

        req.setEmail("emin@test.com");

        req.setPassword("123456");
when(userRepository.existsByEmail("emin@test.com")).thenReturn(true);

        ResponseStatusException ex =
assertThrows(ResponseStatusException.class,

                () -> authService.register(req));

        assertEquals(HttpStatus.CONFLICT,
ex.getStatusCode());

        assertEquals("Email already exists",
ex.getReason());

        verify(userRepository,
times(1)).existsByEmail("emin@test.com");

        verify(userRepository,
never()).save(any(User.class));

        verify(passwordEncoder,
never()).encode(anyString());
    }

    @Test
    void registerAdminShouldSaveUserWithAdminRole() {

        RegisterRequest req = new RegisterRequest();

        req.setName("Admin");

        req.setEmail("admin@test.com");

        req.setPassword("adminpass");
when(userRepository.existsByEmail("admin@test.com")).thenReturn(false);

when(passwordEncoder.encode("adminpass")).thenReturn("bcrypt_admin");

```



```

        authService.registerAdmin(req);

        ArgumentCaptor<User> captor =
ArgumentCaptor.forClass(User.class);

        verify(userRepository,
times(1)).save(captor.capture());

        User saved = captor.getValue();

        assertEquals("Admin", saved.getName());

        assertEquals("admin@test.com", saved.getEmail());

        assertEquals("bcrypt_admin", saved.getPassword());

        assertEquals(UserRole.ROLE_ADMIN, saved.getRole());

        assertNotNull(saved.getCreatedAt());

        verify(passwordEncoder,
times(1)).encode("adminpass");

        verify(userRepository,
times(1)).existsByEmail("admin@test.com");
    }

```

Часть 6. verify(..., times(1))

Пункт выполнен внутри тестов UserServiceDeleteTest и AuthServiceTest (явный times(1) использован).

Часть 7. spy() на список и проверка add()

Методичка объясняет spy как обёртку над реальным объектом с возможностью verify.

Листинг: src/test/java/org/example/SpyListTest.java

```

class SpyListTest {
    @Test
    void shouldSpyListAndVerifyAdd() {
        List<String> list = spy(new ArrayList<>());
        list.add("one");
        list.add("two");
        verify(list, times(1)).add("one");
        verify(list, times(1)).add("two");
        verify(list, times(2)).add(anyString());
    }
}

```

Часть 10. Web-тест контроллера через @WebMvcTest

Методичка показывает @WebMvcTest + MockMvc + @MockBean (в новых версиях может подсвечиваться как deprecated, но логика остаётся прежней).

Важно: @WebMvcTest поднимает только web-слой, поэтому зависимости контроллера подменяются моками через @MockBean (или современный @MockitoBean).

Листинг: src/test/java/org/example/controller/NotificationControllerWebTest.java

```

@WebMvcTest(NotificationController.class)
class NotificationControllerWebTest {
    @Autowired
    private MockMvc mockMvc;

    @MockitoBean
    private NotificationService notificationService;
}

```

```

@Test
void shouldReturnOkForAllNotifications() throws
Exception {
    mockMvc.perform(get("/notifications/all"))
        .andExpect(status().isOk());
}
}

```

Результаты

1. Подключён тестовый стек Spring Boot (JUnit + Mockito и инфраструктура тестирования).
2. Реализованы unit-тесты сервисного слоя: позитивные и негативные сценарии с `assertThrows`, изоляция через `@Mock`/`@InjectMocks`.
3. Реализованы проверки взаимодействий через `verify(..., times(1))`.
4. Реализован пример `spy()` и проверка вызовов реального объекта.
5. Выполнен web-тест контроллера с `@WebMvcTest` и `MockMvc`.
6. Все тесты запускаются общей командой:

```
mvn clean test
```

Вывод

В ходе работы освоены базовые подходы тестирования Spring Boot-приложений: написание unit-тестов для сервисного слоя с полной изоляцией зависимостей через Mockito (`@Mock`/`@InjectMocks`, `when/thenReturn`), проверка взаимодействий (`verify`, `times`), применение `spy()` для частичного мокирования реальных объектов, а также создание минимального web-теста контроллера с `@WebMvcTest` и `MockMvc`.

Ответы на контрольные вопросы

1. Что такое unit-тест?

Unit-тест проверяет один метод/класс изолированно, без реальной БД/сети/контекста, быстро и детерминированно.

2. Чем unit-тест отличается от интеграционного?

Unit - один класс и моки; интеграционный - совместная работа нескольких компонентов, может поднимать Spring-контекст и инфраструктуру.

3. Что делает аннотация @Test?

Помечает метод как тестовый - JUnit обнаруживает и запускает его как отдельный тест-кейс.

4. Для чего assertEquals() и assertThrows()?

assertEquals сравнивает ожидаемое и фактическое значение; assertThrows проверяет, что код выбрасывает ожидаемое исключение.

5. Что делают @BeforeEach и @AfterEach?

@BeforeEach выполняется перед каждым тестом для подготовки данных; @AfterEach - после каждого теста для очистки/завершения.

6. Что такое mock-объект?

Mock - заглушка зависимости, имитирующая поведение реального компонента и позволяющая проверять взаимодействие с ним.

7. Чем mock отличается от stub?

Mock - сам поддельный объект; stub - заранее заданное поведение mock-объекта (например when(...).thenReturn(...)).

8. Для чего используется Mockito?

Для изоляции тестируемого класса: заменить зависимости моками, задать возвращаемые значения и проверить факты вызовов.

9. Что делают @Mock и @InjectMocks?

@Mock создаёт мок-объект; @InjectMocks создаёт тестируемый объект и внедряет в него моки как зависимости.

10. Зачем нужен verify()?

Чтобы проверить, что метод зависимости действительно был вызван (и при необходимости - сколько раз).

11. Что такое spy()?

Spy оборачивает реальный объект: реальная логика сохраняется, но можно проверять вызовы и точно подменять методы.

12. Почему для unit-тестов сервисов удобно mock-репозитории?

Потому что сервис тестируется без реальной базы: репозиторий заменяется моком, а тест проверяет бизнес-логику сервиса, а не инфраструктуру.

13. Что делает @WebMvcTest?

Поднимает только web-слой (контроллеры и MVC-инфраструктуру) без полного контекста приложения.

14. Для чего нужен MockMvc?

Чтобы отправлять HTTP-запросы к контроллеру “внутри теста” без запуска реального сервера и проверять статус/ответ.

15. Почему unit-тесты должны быть изолированными и быстрыми?

Чтобы их можно было запускать часто (после каждого изменения), и чтобы результат зависел только от кода, а не от БД/сети/окружения.

